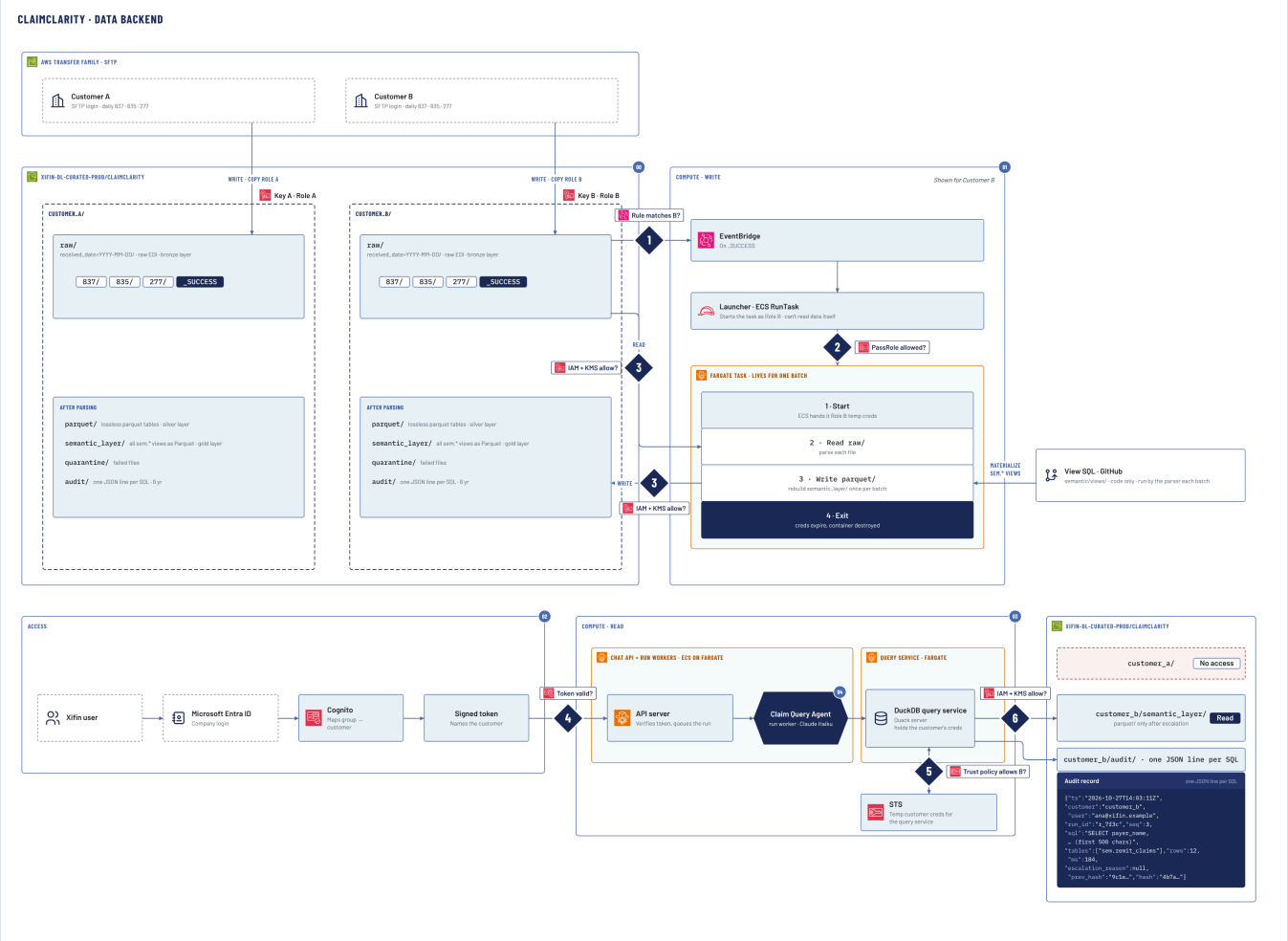
ClaimClarity · Data backend architecture

How customer claim data moves through the ClaimClarity backend, and how each customer's data is kept separate.

Overview



Summary:

This board shows the whole ClaimClarity data backend on one page, drawn for Customer B. Customer A has exactly the same setup with its own key and roles ("Key A · Role A" next to "Key B · Role B"). The board has three lanes: Compute · Write (getting files in and turning them into tables), Access (signing a Xifin user in and working out which customer they belong to), and Compute · Read (answering that user's questions). The numbered diamonds 1–6 are AWS permission checks. Each one sits on the arrow it guards and means the same thing on every board.

On the write side, each customer logs in to AWS Transfer Family over SFTP and drops its daily 837, 835 and 277 files into its own `raw/` folder, under `received_date=YYYY-MM-DD/`. The upload ends with a `_SUCCESS` file. That file fires an EventBridge rule (gate 1). The launcher then calls ECS RunTask and starts a parser task as Role B (gate 2). The launcher can only hand over the role. It can't read any data itself. The parser is a Fargate task that

lives for one batch. ECS gives it temporary credentials for Role B, and it parses each file in `raw/`, writes the lossless tables to `parquet/` (silver) and rebuilds `semantic_layer/` (gold) once per batch from the view SQL kept in GitHub. Then it exits, its credentials expire and the container is destroyed. Every read and write it makes passes gate 3 (IAM + KMS). The design doc gives the reason for this short life: compute should only touch one customer's data and should stop once the work is done, so nothing keeps a connection to PHI open.

On the read side, the Xifin user signs in through Microsoft Entra ID. Cognito maps their group to one customer and issues a signed token that names that customer. The API server checks that token (gate 4) and queues the run. The Claim Query Agent runs in the run worker on Claude Haiku and has no data access of its own. It sends SQL to the DuckDB query service (a Quack server). That service is the only part that holds credentials for the customer. It gets them from STS as temporary credentials for one customer (gate 5), and IAM + KMS allow it to read only Customer B's data (gate 6). It reads `semantic_layer/` normally, reads `parquet/` only after an escalation, and has no access to `customer_a/`. It writes one JSON line per SQL statement to `customer_b/audit/`.

Separation is enforced by AWS (IAM roles, KMS keys, STS, EventBridge rules), not by application code. To touch a customer's data, a process needs that customer's role and that customer's key. The parts that talk to the user (the API server and the agent) never hold either.

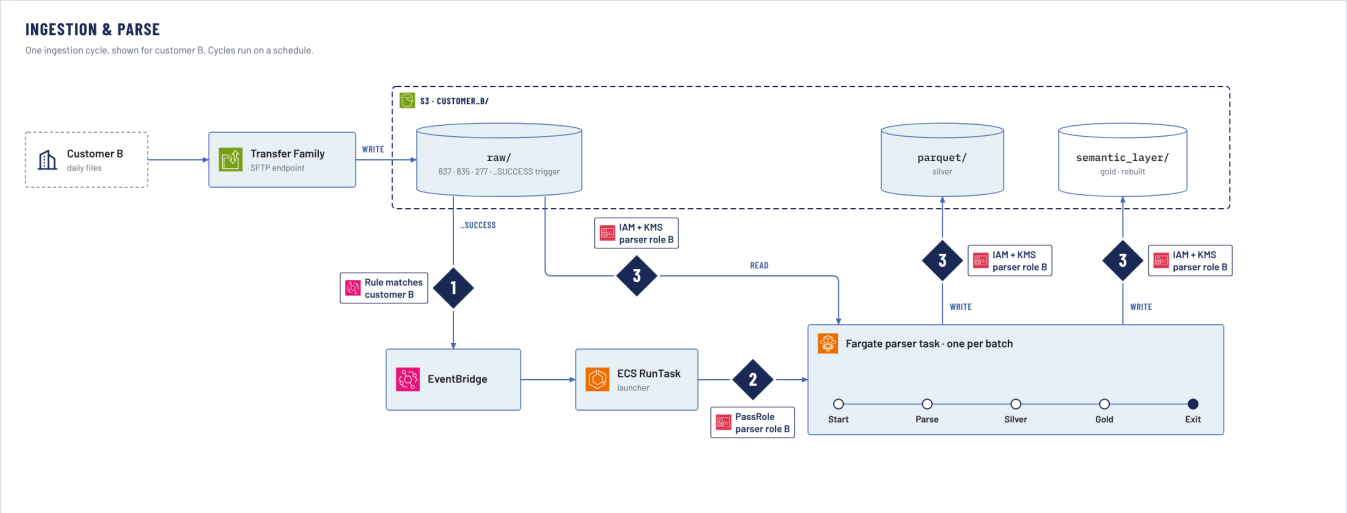
Key components:

- **AWS Transfer Family (SFTP)**: The managed SFTP intake. Each customer logs in and drops its daily 837, 835 and 277 files, and Transfer Family writes them into that customer's `raw/` folder as the customer's copy role ("Write · copy role A/B").
- **Data bucket** `xifin-dl-curated-prod/claimclarity`: One S3 bucket for all customer data, with one prefix per customer (`customer_a/`, `customer_b/`). Each prefix holds `raw/` (bronze), `parquet/` (silver), `semantic_layer/` (gold), `quarantine/` and `audit/`, and has its own KMS key and roles.
- **EventBridge**: Fires when a `_SUCCESS` file lands in a customer's `raw/`. Its rule must match that customer (gate 1, "Rule matches B?") before it calls the launcher.
- **Launcher (ECS RunTask)**: Starts the parser task as Role B after gate 2 ("PassRole allowed?"). It can pass the role on but can't read data itself.
- **Fargate parser task**: A container that lives for one batch, in four steps: 1 · Start (ECS hands it Role B temporary credentials), 2 · Read `raw/` and parse each file, 3 · Write `parquet/` and rebuild `semantic_layer/` once per batch, 4 · Exit (credentials expire, container destroyed). Its reads and writes pass gate 3 ("IAM + KMS allow?").
- **View SQL (GitHub,** `semantic/views/` **)**: Version-controlled SQL that defines the `sem.*` views. It is code only. The parser runs it each batch and materializes the views as Parquet in `semantic_layer/`.
- **Microsoft Entra ID**: The company login the Xifin user signs in with. It sends the user on to Cognito.
- **Cognito**: Maps the user's group to a customer and issues a signed token that names that customer. This token is the only place the backend learns which customer the user belongs to.
- **API server (Chat API + run workers, ECS on Fargate)**: Verifies the token (gate 4, "Token valid?") and queues the run on a run worker.
- **Claim Query Agent**: The run worker itself, running Claude Haiku. It writes the SQL for a question but has no access to the data. It sends the SQL to the DuckDB query service and gets rows back.
- **DuckDB query service (Quack server, Fargate)**: The only component that holds the customer's credentials and reads data. It reads `customer_b/semantic_layer/`, reads `parquet/` only after escalation, and has "No access" to `customer_a/`.

- **STS**: Issues temporary credentials for one customer to the query service, after the role's trust policy allows Customer B (gate 5). IAM + KMS then limit what those credentials can read (gate 6).
- `customer_b/audit/`: The query audit. It holds one JSON line per SQL statement, with timestamp, customer, user, run id, SQL (first 500 characters), tables, rows, timing, escalation reason and a hash chain (`prev_hash`, `hash`).

Notes / TODO:

01 · Ingestion & parse



Summary:

This board follows one ingestion cycle for Customer B, from a file upload to fresh tables. The board notes that cycles run on a schedule. Customer B sends its daily files to the Transfer Family SFTP endpoint, which writes them into `customer_b/raw/` along with a `_SUCCESS` file. `_SUCCESS` is the signal that the batch is complete, and it's what starts everything after it.

When `_SUCCESS` lands, EventBridge passes it on, but only if the rule matches Customer B (gate 1). Each customer has its own rule, so one customer's upload can't start another customer's parser. The rule calls the ECS RunTask launcher, which starts a parser task with parser role B (gate 2). The launcher's IAM policy (`iam:PassRole`) lets it pass only this customer's parser role, and only to ECS. So the launcher can start the right task, but it can't give that task any other customer's role, and it can't read the data itself.

The Fargate parser task runs once per batch: Start, Parse, Silver, Gold, Exit. It reads the 837, 835 and 277 files from `raw/`, writes the parsed lossless tables to `parquet/` (silver), and then rebuilds `semantic_layer/` (gold) once for the whole batch rather than once per file. Every arrow between the task and S3 passes gate 3: parser role B needs the IAM permission on Customer B's prefix and use of Customer B's KMS key for each read and write. Then the task exits.

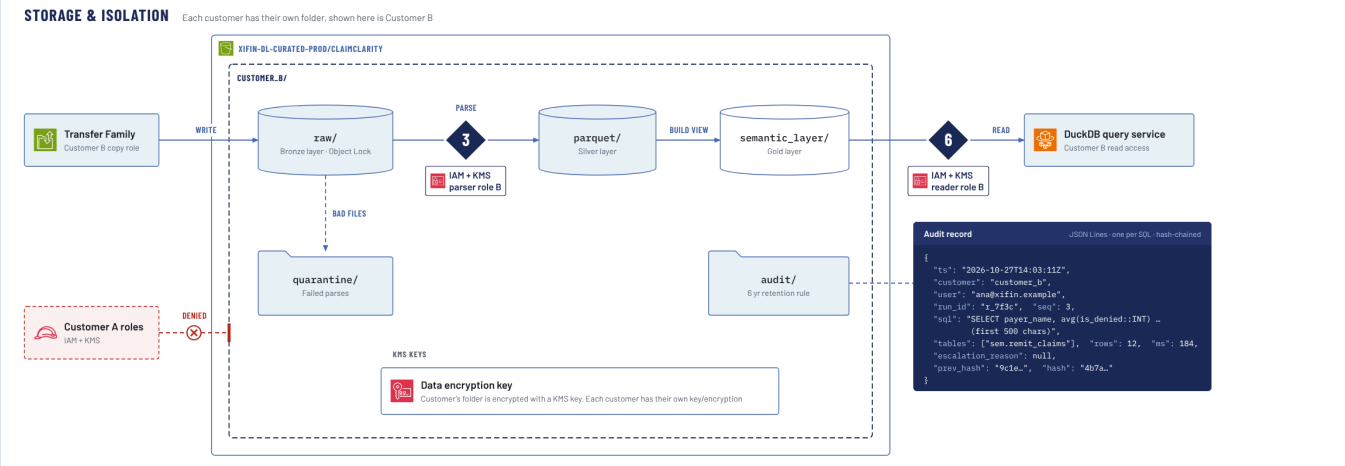
The task is short-lived and holds only one customer's role. The design doc says this is deliberate: the compute should touch only that customer's data and should end when the work is done, so no process keeps a connection to PHI open. Customer A runs the same chain in a separate task with its own role.

Key components:

- **Customer B**: The customer's system that sends daily 837 (claims), 835 (remittances) and 277 files. It is the start of the cycle.
- **Transfer Family SFTP endpoint**: The managed SFTP server. It writes the incoming files and the `_SUCCESS` trigger into `customer_b/raw/` (the "Write" arrow).
- `raw/` : Customer B's bronze folder in S3, holding the 837, 835 and 277 files plus the `_SUCCESS` trigger. The parser reads from it.
- **EventBridge**: Reacts to the `_SUCCESS` file and calls the launcher. Gate 1 ("Rule matches customer B") means the rule fires only for Customer B's folder.
- **ECS RunTask launcher**: Starts the parser task. Gate 2 ("PassRole parser role B") limits it to passing only parser role B, and only to ECS.
- **Fargate parser task (one per batch)**: One container per batch that runs Start → Parse → Silver → Gold → Exit. All three of its S3 arrows (read `raw/` , write `parquet/` , write `semantic_layer/`) pass gate 3 ("IAM + KMS parser role B").
- `parquet/` **(silver)**: The lossless Parquet tables parsed from the raw EDI, written by the parser task.
- `semantic_layer/` **(gold)**: The `sem.*` views materialized as Parquet, "rebuilt" by the parser at the end of each batch. This is the layer the query side reads.

Notes / TODO:

00 · Storage & isolation



Summary:

This board zooms in on one customer's folder in the shared bucket `xifin-dl-curated-prod/claimclarity` , shown for Customer B. Every customer gets its own folder, its own KMS data encryption key and its own set of roles. That gives two locks on every piece of data. To read anything in `customer_b/` , a process needs a role whose IAM policy reaches that prefix and permission to use Customer B's key. Either one alone gets it nothing.

The folder follows a bronze, silver, gold layout, and each layer has one writer. Transfer Family writes raw EDI into `raw/` as the Customer B copy role. `raw/` is under Object Lock, so files can't be changed once they land and the raw data stays the source of truth. The parser (parser role B, through gate 3) reads `raw/` , writes the lossless tables to `parquet/` (silver), builds the views into `semantic_layer/` (gold), and sends bad files to `quarantine/` . On the read side, the DuckDB query service reads `semantic_layer/` with Customer B read access, through gate 6 (IAM + KMS for reader role B).

`audit/` keeps a record of every query: one JSON Lines record per SQL statement, hash-chained, under a 6-year retention rule. Each record holds the hash of the record before it (`prev_hash`) and its own `hash` , so the records form a chain. The example below shows the fields.

Customer A's roles are drawn on the right with a "Denied" arrow. IAM and KMS both block them from Customer B's folder, so a mistake in one customer's setup can't expose another customer's data.

Key components:

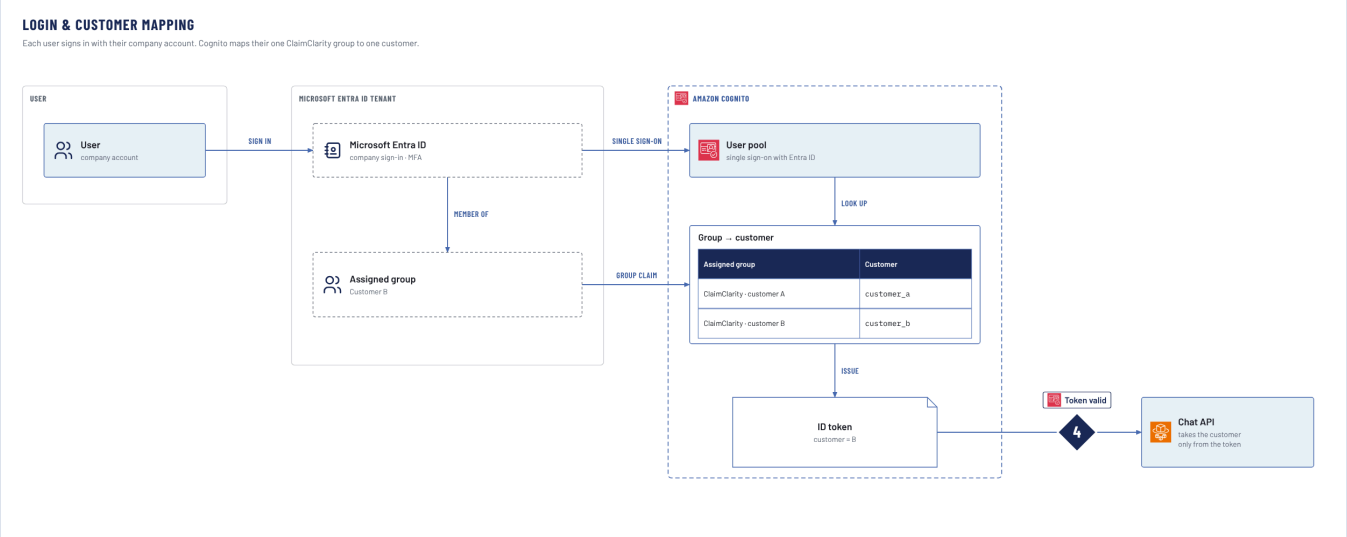
- `raw/` : The bronze layer, raw EDI exactly as received, under Object Lock (write once). It is written by the Customer B copy role through Transfer Family and read by the parser.
- `parquet/` : The silver layer of lossless Parquet tables, written by parser role B during the parse step.
- `semantic_layer/` : The gold layer. Parser role B builds it from the view SQL ("Build view"). The DuckDB query service reads it with Customer B read access.
- `quarantine/` : Holds failed parses. The parser writes files it couldn't parse here ("Bad files").
- `audit/` : The query audit trail. It holds one JSON Lines record per SQL statement, hash-chained, with a 6-year retention rule.
- **Data encryption key (KMS)**: Customer B's own KMS key, which encrypts the customer's folder. Each customer has their own key, so reading or writing needs permission on that key as well as the IAM role.
- **Gate 3**: IAM + KMS for parser role B. It guards the parser's read of `raw/` and its writes to `parquet/` , `semantic_layer/` and `quarantine/` .
- **Gate 6**: IAM + KMS for reader role B. It guards the DuckDB query service's read of `semantic_layer/` (Customer B read access).
- **Customer A roles**: Drawn outside the folder with a "Denied" arrow. They are blocked from `customer_b/` by both IAM and KMS.

Example audit record (one line per SQL):

```
{"ts": "2026-10-27T14:03:11Z", "customer": "customer_b", "user": "ana@xifin.example",
 "run_id": "r_7f3c", "seq": 3, "sql": "SELECT payer_name, … (first 500 chars)",
 "tables": ["sem.remit_claims"], "rows": 12, "ms": 184,
 "escalation_reason": null, "prev_hash": "9c1e…", "hash": "4b7a…"}
```

Notes / TODO:

02 · Login & customer mapping



Summary:

This board shows how a user's login becomes "this user may see Customer B's data". The user signs in with their company account at Microsoft Entra ID, which handles the company sign-in and MFA. In Entra ID the user is a member of one assigned group, here "ClaimClarity · customer B". Admins grant access by adding the user to that group, so there is no separate user table to keep in sync.

Entra ID passes the user to the Amazon Cognito user pool through single sign-on, along with the group claim. Cognito looks the group up in a group → customer table ("ClaimClarity · customer A" → `customer_a`, "ClaimClarity · customer B" → `customer_b`) and issues an ID token that carries `customer = B`. That gives two points where a user can be turned away: Entra ID doesn't let them sign in, or their group doesn't map to a customer.

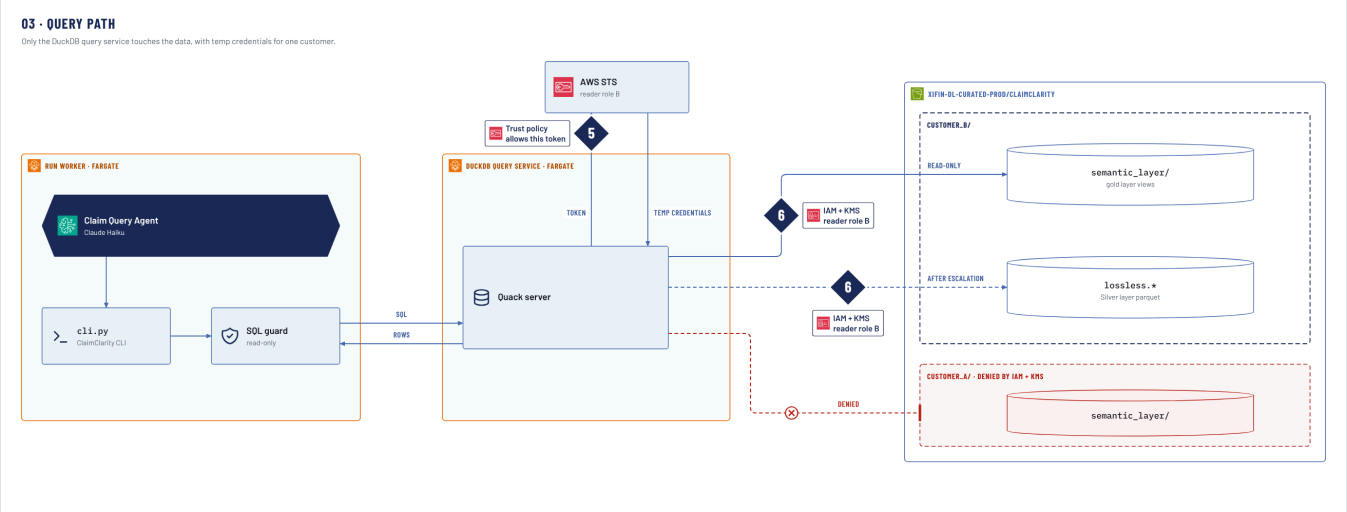
The Chat API accepts a request only if the token is valid (gate 4) and takes the customer only from the token. The customer never comes from the URL or the request body. A user can't ask for another customer's data by changing a request, because the token decides the customer and Cognito signs it.

Key components:

- **User**: A Xifin user signing in with their company account. This is the start of the "Sign in" arrow.
- **Microsoft Entra ID**: The company identity provider, which does sign-in with MFA. The user is a "Member of" one assigned group (e.g. Customer B), and that group is sent to Cognito as the group claim.
- **Cognito user pool**: The app's Amazon Cognito user pool, set up for single sign-on with Entra ID. It reads the group claim, looks it up in the group → customer table and issues the ID token.
- **Group → customer table**: The lookup inside Cognito that maps "ClaimClarity · customer A" → `customer_a` and "ClaimClarity · customer B" → `customer_b`.
- **ID token**: The signed token Cognito issues. It carries `customer = B` and is the only source of the customer for the rest of the system.
- **Chat API**: The backend API. It checks the token on every request (gate 4, "Token valid") and takes the customer only from the token.

Notes / TODO:

03 · Query path



Summary:

This board shows how a question turns into rows from Customer B's data, and why the agent never touches the data itself. The work is split across two Fargate services. The run worker holds the Claim Query Agent (Claude Haiku), `cli.py` (the ClaimClarity CLI) and the read-only SQL guard. The DuckDB query service runs the Quack server. Only the query service can reach S3, and only with temporary credentials for one customer.

The agent writes SQL and sends it through `cli.py`. The SQL guard checks it, then it travels to the Quack server. The query service passes the user's token to AWS STS. STS returns temporary credentials for reader role B only if the role's trust policy allows this token (gate 5). With those credentials the query service reads Customer B's `semantic_layer/` (gold layer views, read-only). It reads the silver-layer `lossless.*` Parquet only after the agent has escalated. Both reads pass gate 6 (IAM + KMS for reader role B). The rows go back to the agent.

The agent and run worker hold no AWS credentials, and the data credentials are short-lived and tied to one customer. So even if the agent wrote SQL that pointed at `customer_a/`, the read would fail: IAM + KMS deny reader role B there, whatever the application code does. Customer A is blocked twice, once because the API takes the customer only from the token and once by AWS permissions on the bucket.

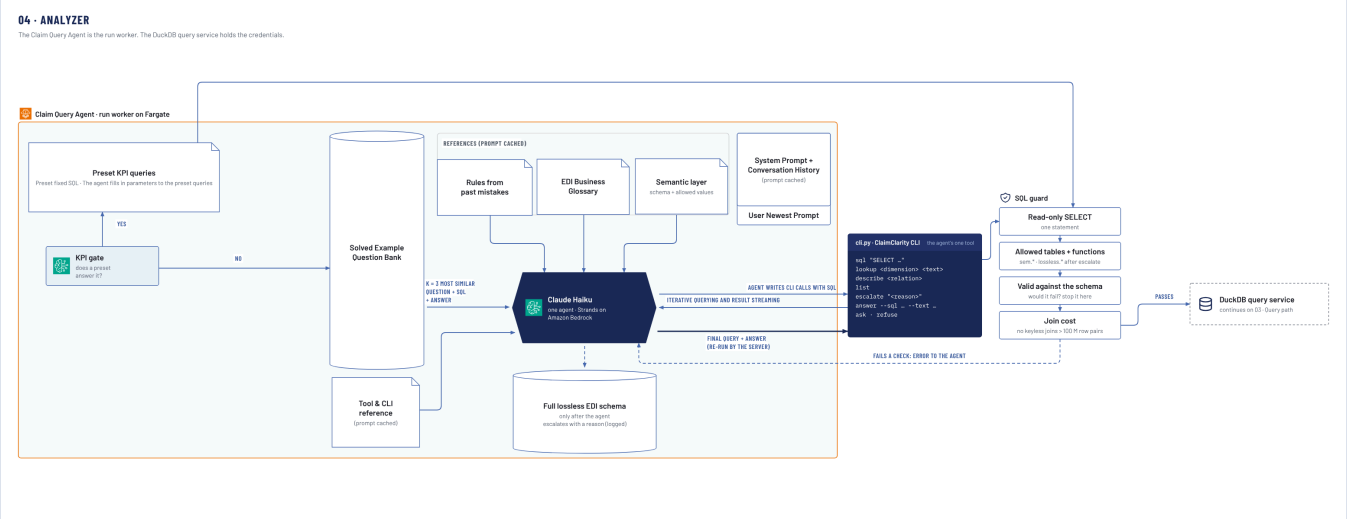
Key components:

- **Run worker (Fargate)**: The Fargate service that holds the Claim Query Agent (Claude Haiku), `cli.py` (ClaimClarity CLI) and the read-only SQL guard. It sends SQL out and gets rows back, but holds no data credentials.
- **Quack server (DuckDB query service, Fargate)**: DuckDB running as its own Fargate service. It receives SQL from the run worker, sends the token to STS, reads S3 with the temporary credentials and returns rows.
- **AWS STS**: Exchanges the user's token for temporary credentials for reader role B. Gate 5 ("Trust policy allows this token") is the reader role's trust policy, which accepts only this token's customer.
- `semantic_layer/`: Customer B's gold layer views, read-only. The normal source for answers. Gate 6 (IAM + KMS for reader role B) guards the read.
- `lossless.*`: Customer B's silver-layer Parquet tables. The query service reads them only "after escalation", and the read also passes gate 6.

- `customer_a/` : Another customer's folder in the same bucket. Reads with reader role B credentials are denied by IAM + KMS.

Notes / TODO:

04 · Analyzer



Summary:

This board shows how the Claim Query Agent turns a user's question into SQL. The Claim Query Agent is the run worker itself, and the DuckDB query service holds the credentials. A new question first reaches the KPI gate, which asks "does a preset answer it?". If it does ("Yes"), the question runs as one of the preset KPI queries: fixed SQL with its parameters filled in for this question. If not ("No"), the question goes to the agent.

The agent is one Claude Haiku agent, built with Strands on Amazon Bedrock. It gets the system prompt and conversation history (prompt cached), the user's newest prompt, and a set of prompt-cached references: the preset KPI queries, the tool and CLI reference, rules from past mistakes, an EDI business glossary, and the semantic layer schema with its allowed values. From the solved-example question bank it also gets the K = 3 most similar past questions, each with its SQL and answer. The full lossless EDI schema is held back. The agent gets it only after it escalates with a reason, and the reason is logged. So by default the agent works on the curated semantic layer, and every use of the lossless tables leaves a recorded reason (the `escalation_reason` field in the audit record).

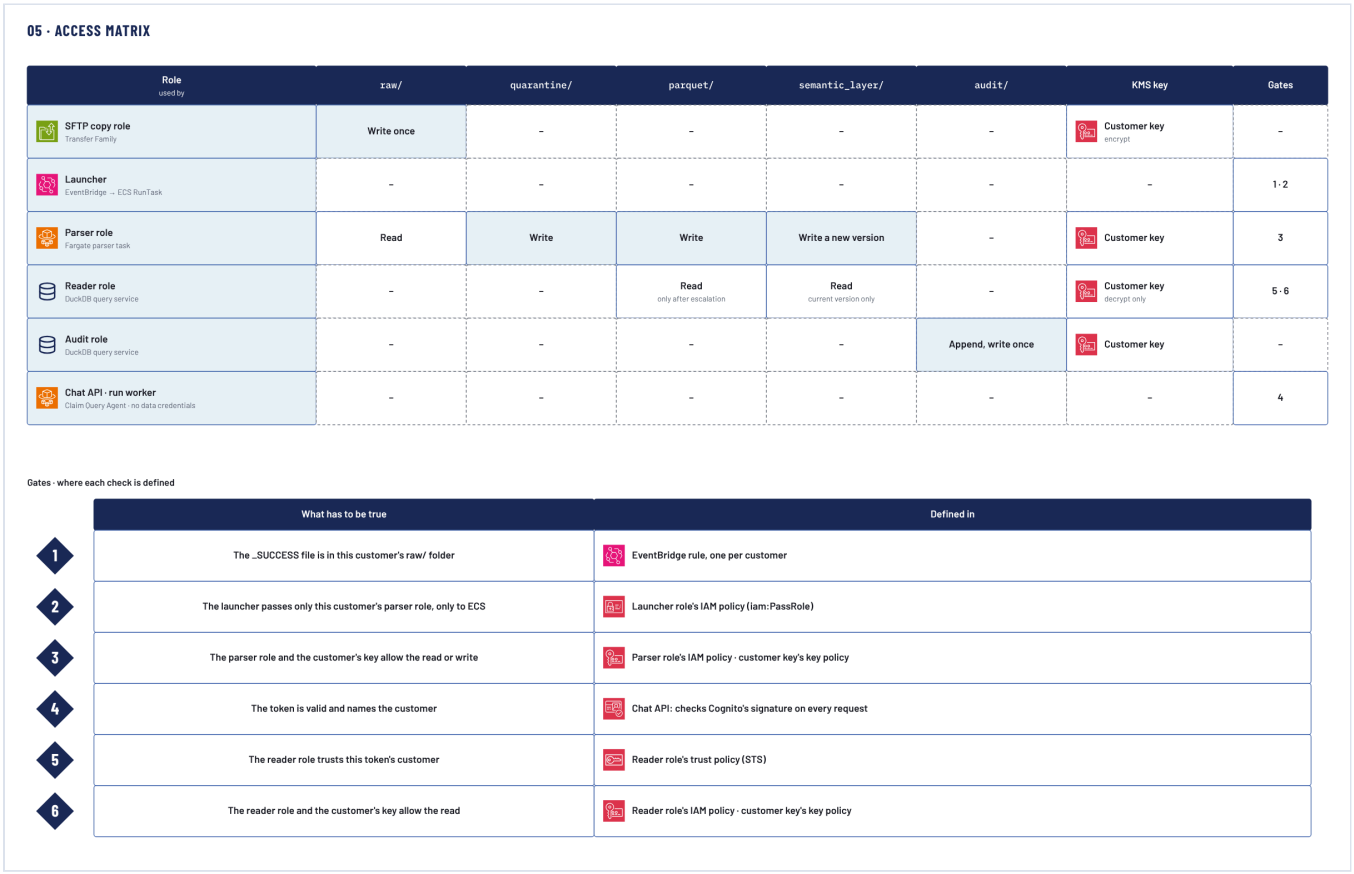
The agent acts only through its one tool, `cli.py`. Every SQL call goes through the SQL guard before the DuckDB query service. The guard checks four things: the SQL is a read-only SELECT with one statement; it uses only allowed tables and functions (`sem.*`, plus `lossless.*` after escalation); it is valid against the schema ("would it fail? stop it here"); and its join cost is acceptable (no keyless joins over 100 M row pairs). A call that passes goes on to the query service (continued on 03 · Query path). A call that fails a check goes back to the agent as an error, so bad SQL never reaches the data. The agent queries step by step and the results stream back. When it's done, it returns a final query and answer, and the server re-runs that final query so the numbers come from the database, not from the model.

Key components:

- **KPI gate**: The first decision for every new prompt: "does a preset answer it?". "Yes" runs a preset KPI query and "No" hands the question to the Claim Query Agent.
- **Preset KPI queries**: Preset fixed SQL with parameters filled in. They are also part of the agent's prompt-cached references.
- **Claim Query Agent**: One agent on Claude Haiku, built with Strands on Amazon Bedrock and running as the run worker on Fargate. Its inputs are the system prompt and conversation history (prompt cached), the user's newest prompt and the references.
- **References (prompt cached)**: The preset KPI queries, the tool and CLI reference, rules from past mistakes, the EDI business glossary, and the semantic layer schema with allowed values. The agent also gets the K = 3 most similar question + SQL + answer from the solved-example question bank.
- **Full lossless EDI schema**: Drawn as an optional input. The agent gets it only after it escalates with a reason, and that reason is logged.
- `cli.py` **(ClaimClarity CLI)**: The agent's one tool. Its commands are `sql "SELECT …"`, `lookup <dimension> <text>`, `describe <relation>`, `list`, `escalate "<reason>"`, `answer --sql … --text …`, `ask` and `refuse`.
- **SQL guard**: Checks every SQL call before it leaves the run worker: read-only SELECT (one statement); allowed tables and functions (`sem.*`, `lossless.*` after escalate); valid against the schema; join cost (no keyless joins over 100 M row pairs). "Passes" goes to the DuckDB query service, and "Fails a check" returns an error to the agent.
- **DuckDB query service**: The service that holds the credentials and runs the guarded SQL. The flow continues on 03 · Query path.

Notes / TODO:

05 · Access matrix



Summary:

This board is a reference table rather than a flow. It lists every role in the backend against every folder in one customer's prefix (`raw/`, `quarantine/`, `parquet/`, `semantic_layer/`, `audit/`), the customer's KMS key and the gates that guard it. Each role can do only the one job its part of the flow needs. The SFTP copy role can only write raw files once. The parser role turns raw into silver and gold. The reader role can only read. The audit role can only append audit records.

Some parts have no data access at all. The launcher only starts the parser. The Chat API and run worker, where the Claim Query Agent runs, hold no data credentials. They are guarded by gates 1–2 and gate 4, not by S3 permissions. Two cells encode rules from the other boards. The reader reads `parquet/` only after escalation, and it reads only the current version of `semantic_layer/`, while the parser writes each rebuild as a new version.

The second table lists the six gates: what has to be true at each one and where that check is configured. That makes the board useful for reviewing the real IAM, KMS, STS and EventBridge setup.

Key components:

- **SFTP copy role (Transfer Family)**: Writes once to `raw/` and has no access to the other folders. It may only encrypt with the customer key.
- **Launcher (EventBridge → ECS RunTask)**: Has no access to any folder or the key. It is guarded by gates 1 and 2.
- **Parser role (Fargate parser task)**: Reads `raw/`, writes `quarantine/` and `parquet/`, and writes a new version of `semantic_layer/`. It has no access to `audit/`, uses the customer key and is guarded by gate 3.
- **Reader role (DuckDB query service)**: Reads `parquet/` only after escalation and reads only the current version of `semantic_layer/`. It has no access to `raw/`, `quarantine/` or `audit/`, may only decrypt with the customer key, and is guarded by gates 5 and 6.
- **Audit role (DuckDB query service)**: Can append, write once, to `audit/` and to nothing else. It uses the customer key.
- **Chat API · run worker (Claim Query Agent)**: Holds no data credentials and has no access to any folder or the key. It is guarded by gate 4.

The six gates:

- **Gate 1**: The `_SUCCESS` file is in this customer's `raw/` folder. Defined in the EventBridge rule, one per customer.
- **Gate 2**: The launcher passes only this customer's parser role, and only to ECS. Defined in the launcher role's IAM policy (`iam:PassRole`).
- **Gate 3**: The parser role and the customer's key allow the read or write. Defined in the parser role's IAM policy and the customer key's key policy.
- **Gate 4**: The token is valid and names the customer. The Chat API checks Cognito's signature on every request.
- **Gate 5**: The reader role trusts this token's customer. Defined in the reader role's trust policy (STS).
- **Gate 6**: The reader role and the customer's key allow the read. Defined in the reader role's IAM policy and the customer key's key policy.

Notes / TODO: